ZEEVAN

AI Health Companion · Technical Report

Generated: 25 October 2026, 19:23
Confidential — Internal Project Documentation

Document Type	Technical Specification & Evaluation Report
Project	ZEEVAN — AI-Powered Health Companion (Phases 1–3)
Report Version	1.0
Date	25 October 2026
Classification	Internal / Academic Submission
Stack	React + Vite (Frontend) · FastAPI + PostgreSQL (Backend) · scikit-learn (ML)

This document provides a complete technical record of the ZEEVAN health-companion application as submitted for milestone review. It covers the full NLP and machine-learning pipeline, dataset provenance and preparation, training metrics, live backend integration evidence, and an honest assessment of model limitations.

1 · Executive Summary

ZEEVAN is a three-phase project delivering a privacy-focused AI health companion. Users describe symptoms in natural language (including Hindi/Hinglish), receive AI-powered symptom normalisation, structured follow-up questions, a Random Forest disease-pattern prediction, and safety-flagging — all backed by a FastAPI/PostgreSQL REST API and a React/Vite single-page application.

Metric	Value
Application phases completed	3 of 3
Backend API endpoints	11 REST routes
Disease classes predicted	41
Symptom features (binary)	133 + 1 engineered
Training samples	4 920 rows
Holdout accuracy (stratified 80/20)	100 % *
Macro F1 score	1.00 *
Model artifact size	~18.2 MB (joblib)
Frontend framework	React 19 + Vite 7
Backend framework	FastAPI 0.115 + Uvicorn
Database	PostgreSQL via SQLAlchemy 2.0

■ ** The 100 % holdout accuracy is a known artefact of the source dataset containing near-duplicate rows with deterministic symptom-to-disease mappings. It reflects pattern memorisation, NOT clinical generalisation. See Section 5.*

2 · System Architecture

ZEEVAN uses a decoupled three-tier architecture:

Tier	Technology	Responsibility
Presentation	React 19 · Vite 7 · Framer Motion · Lucide React	SPA routing, symptom composer, voice input (Web Spe
API Gateway	FastAPI 0.115 · Uvicorn · Pydantic v2	JWT auth, request validation, orchestration of NLP → M
Persistence	PostgreSQL · SQLAlchemy 2.0 · joblib	User accounts, symptom checks, follow-up Q&A, health

2.1 Request Lifecycle

- User submits symptom text + optional chip selections via POST /api/symptoms/check
- NLP layer (nlp.py) normalises free text → canonical symptom tokens
- Backend persists SymptomCheck row; returns 3 structured follow-up questions
- User answers are POSTed to /api/symptoms/check/{id}/analyze
- ML layer (predict.py) encodes symptom vector → RandomForest.predict_proba → top-3 predictions
- Safety engine checks description for urgent phrases and sets urgent=True if matched
- Full result JSON returned to frontend and persisted (result_json column)

2.2 Authentication

- PBKDF2-SHA256 with 120 000 iterations for password hashing (no third-party bcrypt dependency)
- JWT (HS256) tokens signed with JWT_SECRET env var; default expiry 24 hours
- All health-journey endpoints require Bearer token; symptom check works anonymously

3 · NLP Pipeline for Symptom Analysis

The NLP layer (backend/app/nlp.py) is a purpose-built, rule-based symptom extractor optimised for the specific vocabulary of the training dataset. This design was chosen for determinism, speed, and zero external API dependency.

3.1 Library & Language Stack

Library / Module	Version	Role in Pipeline
Python re (stdlib)	3.12	Regex tokenisation and phrase matching
pandas	2.2.3	Dataset loading, column cleaning, feature frame construction
NumPy	2.2.1	Numerical array operations backing scikit-learn
scikit-learn	1.6.0	RandomForestClassifier, train/test split, metrics
joblib	1.4.2	Model serialisation / deserialisation (pickle-safe)
FastAPI / Pydantic v2	0.115 / 2.10	API layer; Pydantic validates & coerces inputs before NLP
Web Speech API	Browser native	Voice-to-text on frontend (no server-side ASR)

3.2 Symptom Extraction Algorithm

The extract_symptoms() function in nlp.py implements a two-stage pipeline:

- Stage 1 — Text normalisation: input is lowercased and non-alphanumeric characters are replaced with spaces using re.sub(r'[^a-z0-9\s]', ' ', text.lower()).
- Stage 2 — Alias matching: for each of 14 canonical symptom tokens, a list of natural-language phrases (aliases) is checked against the normalised text using word-boundary regex (\b{phrase}\b). A canonical token is added to the found set if any alias matches.
- Stage 3 — Chip merge: user-selected UI chips are cleaned with preprocess.clean_name() (snake_case, strip punctuation) and merged into the found set.
- Output: a sorted list of canonical symptom strings passed to the ML layer.

3.3 Canonical Symptom Vocabulary (14 tokens)

Canonical Token	Matched Phrases (examples)
headache	headache, head pain, migraine
fatigue	fatigue, tired, low energy, exhausted
fever	fever, high temperature, temperature
nausea	nausea, feeling sick, sick to my stomach
breathlessness	breathlessness, shortness of breath, difficulty breathing
chest_pain	chest pain, tight chest, pressure in chest
cough	cough, coughing
runny_nose	runny nose, blocked nose, stuffy nose, congestion

Canonical Token	Matched Phrases (examples)
sore_throat	sore throat, throat pain
abdominal_pain	stomach pain, abdominal pain, belly pain
diarrhoea	diarrhea, diarrhoea, loose stools
dizziness	dizzy, dizziness, lightheaded
joint_pain	joint pain, aching joints
skin_rash	rash, skin rash, spots on skin

3.4 Safety / Urgency Detection

A separate urgent_flags() function scans the raw description (not normalised) for four high-priority phrases:

- "chest pain" → flag: chest_pain
- "severe difficulty breathing" → flag: severe_breathing
- "loss of consciousness" → flag: loss_of_consciousness
- "fainted" → flag: fainting

If any flag is raised, the API response sets urgent=True, which triggers a prominent red warning card on the frontend instructing the user to seek emergency care. This is independent of the ML prediction.

3.5 Follow-up Question Engine

After symptom extraction, the API always presents three structured follow-up questions with fixed categorical options. These are persisted to the follow_up_questions table and the user's answers are stored in follow_up_answers before analysis:

Key	Question Text	Options
duration	How long have you had the main symptom?	Less than a day / 1–3 days / More than 3 days
severity	How would you describe the discomfort?	Mild / Moderate / Severe
fever	Do you have a fever?	Yes / No / Not sure

4 · Machine Learning Model

4.1 Algorithm — RandomForestClassifier

A Random Forest ensemble classifier was selected for its robustness to irrelevant features, native multi-class support, and interpretability via feature importances. No deep learning framework is used; the entire inference chain runs in-process with scikit-learn.

Hyperparameter	Value	Rationale
n_estimators	240	Higher tree count → lower variance at moderate cost
random_state	42	Reproducibility
class_weight	balanced_subsample	Compensates for any minor class imbalance per bootstrap
n_jobs	-1	Use all CPU cores during training
train/test split	Stratified 80/20	Preserves class distribution in holdout set
Stratify on	prognosis column	Ensures each class appears in both train and test

4.2 Feature Engineering

- 133 binary symptom columns from the raw CSV (0/1 per symptom per row)
- 1 engineered feature: symptom_count = row-wise sum of all binary symptom columns
- Total feature dimension: 134
- At inference time: the canonical tokens from the NLP layer are matched to the 134 column names; matched columns are set to 1, all others to 0

4.3 Inference Pipeline (predict.py)

The predict() function runs as follows:

```
1. load_bundle() — lazy-loads symptom_model.joblib on first call
2. Build a zero-vector dict keyed by the 134 column names
3. For each NLP-extracted symptom token: if token in column names → set to 1
4. Create a single-row pd.DataFrame
5. Append symptom_count feature
6. model.predict_proba(frame)[0] → probability distribution over 41 classes
7. argsort descending → return top-3 {label, confidence} dicts
```

4.4 Model Artifact

Artifact Key	Content
model	Trained RandomForestClassifier object
symptom_columns	Ordered list of 133 feature column names
feature_columns	Full 134-column list including symptom_count
dataset_source	anujdutt9/Disease-Prediction-from-Symptoms
model_version	phase-2-random-forest-v1

The .joblib file is ~18.2 MB on disk (240 trees × 41 classes × 134 features). It is loaded once at API startup and cached in a module-level variable for zero-latency inference.

5 · Dataset Preparation & Model Training Metrics

5.1 Dataset Provenance

Source: anujdutt9/Disease-Prediction-from-Symptoms (GitHub), which cites the original Kaggle dataset 'Disease Prediction Using Machine Learning' by kaushil268.

File used: data/raw/training_data.csv — retrieved 2026-09-26 for local development.

Property	Value
Total rows	4 920
Disease classes (prognosis)	41
Rows per class (approx.)	120 (highly balanced)
Symptom feature columns	133 binary (0/1)
Engineered features added	1 (symptom_count)
Target column	prognosis (string label)
Missing values strategy	pd.to_numeric(errors='coerce').fillna(0).clip(0,1)
Duplicate column handling	Suffix _2, _3… appended by preprocess.load_dataset()

5.2 Preprocessing Steps (backend/app/ml/preprocess.py)

- Column names cleaned with clean_name(): lowercased, spaces → underscores, non-alphanumeric characters stripped
- Duplicate column names disambiguated by appending _2, _3 suffixes
- All symptom columns coerced to int {0, 1} via pd.to_numeric → fillna(0) → clip(0,1)
- prognosis column stripped of whitespace and cast to str
- symptom_count feature appended in build_features() before training/inference

5.3 Training Metrics (reports/model_evaluation.json)

■ *The following metrics are from the automated evaluation run by train.py. The perfect scores are expected given the dataset structure (see 5.4 Limitations).*

Metric	Value
Holdout set size (20 % stratified)	984 rows
Overall Accuracy	1.0000 (100%)
Macro-average Precision	1.0000
Macro-average Recall	1.0000
Macro F1 Score (macro avg)	1.0000
Weighted Precision	1.0000
Weighted Recall	1.0000

Metric	Value
Weighted F1 Score	1.0000
Per-class support (each of 41 classes)	24 test samples
Per-class Precision / Recall / F1	1.0 / 1.0 / 1.0 for all 41 classes

5.4 Honest Limitations Assessment

The training data is highly synthetic: each disease maps to a near-deterministic combination of symptoms with minimal noise. This means:

- The holdout set contains near-duplicates of training rows → the model effectively memorises the mapping rather than generalising from real patient presentations.
- Accuracy of 100 % on this dataset does not imply clinical reliability.
- Real symptom presentations overlap significantly between diseases; the clean boundaries in this dataset do not reflect clinical reality.
- No external validation set (e.g. real patient records, MIMIC-III) was used.
- The model is explicitly labelled demo=True in every API response to prevent users from treating results as a medical diagnosis.

These limitations are documented in the model artifact (model_evaluation.json, key: 'limitations') and disclosed in the frontend result UI.

6 · Functional Integration Evidence

This section documents the live backend data processing that proves end-to-end functional integration beyond a static mockup.

6.1 Live API Endpoints (11 routes)

Method	Route	Auth	Description
GET	/api/health	None	Liveness probe → {status: ok}
GET	/api/model	None	Returns model metadata from joblib artifact
GET	/api/phases	None	Returns project phase completion status
POST	/api/auth/register	None	Create user; returns JWT access_token
POST	/api/auth/login	None	Verify PBKDF2 hash; return JWT
GET	/api/auth/me	Bearer	Return authenticated user profile
POST	/api/symptoms/check	Optional	NLP normalise → persist → return questions
POST	/api/symptoms/check/{id}/analyze	Optional	Run ML prediction → persist result_json
POST	/api/symptoms/{id}/save	Bearer	Attach check to authenticated user
GET	/api/journeys	Bearer	List user's saved symptom checks
POST	/api/journeys/{id}/notes	Bearer	Append free-text note to journey item

6.2 Database Schema (6 tables)

Table	Key Columns	Purpose
users	id, email, password_hash, full_name	Registered user accounts
symptoms	id, name, category	Normalised symptom vocabulary
symptom_checks	id, user_id, initial_description, normalized_symptoms, result_json, status	Symptom check session
symptom_check_symptoms	symptom_check_id, symptom_id, severity, duration	Join between checks and symptoms
follow_up_questions	id, symptom_check_id, question_key, options_json	Structured Q per check
follow_up_answers	id, follow_up_question_id, answer	User answers stored server-side
health_journey_notes	id, symptom_check_id, note	Free-text user annotations

6.3 Sample API Response (POST /api/symptoms/check/{id}/analyze)

The following is the actual JSON structure returned by the live backend after ML inference. This confirms data flows from user input → NLP → ML model → persisted JSON → frontend display:


```
{ "demo": true,
"urgent": false,
"nlp_extracted_symptoms": ["fatigue", "headache"],
"predictions": [
```

```
{ "label": "Migraine", "confidence": 0.4832 },
{ "label": "Common Cold", "confidence": 0.1941 },
{ "label": "Hypertension", "confidence": 0.0874 }
],
"model": { "available": true, "version": "phase-2-random-forest-v1",
"features": 133, "dataset_source": "anujdutt9/..." },
"explanations": [...], "advice": [...], "safety": [...], "doctor": "..." }
```


6.4 Distinguishing Live Processing from Static Mockup

- Every disease-pattern prediction includes a model.version field populated from the loaded joblib artifact — a static mockup cannot forge this dynamically.
- The nlp_extracted_symptoms field reflects what the NLP engine extracted from the user's exact free text input in real time.
- confidence values are genuine output of RandomForestClassifier.predict_proba() — they vary per input and sum to 1.0 across all 41 classes.
- SymptomCheck rows are written to PostgreSQL with timestamps; the /api/journeys endpoint reads these live records sorted by created_at DESC.
- The /api/model endpoint returns features: 133 directly from bundle['symptom_columns'] read out of the saved artifact — not hardcoded.

7 · Complete Technology Stack

Layer	Technology	Version	Purpose
Frontend UI	React	19.2.1	Component-based SPA
Frontend Build	Vite	7.1.7	Dev server + HMR + bundling
Animations	Framer Motion	12.x	Page transitions, orb animations
Icons	Lucide React	0.453	Icon library
Voice Input	Web Speech API	Native	Browser-based speech-to-text
API Framework	FastAPI	0.115.6	REST API with auto OpenAPI docs
ASGI Server	Uvicorn	0.34.0	Production-grade async server
Data Validation	Pydantic v2	2.10.4	Request/response schema validation
ORM	SQLAlchemy	2.0.36	Database abstraction layer
Database	PostgreSQL	15+	Primary data store
DB Driver	psycopg2-binary	2.9.10	PostgreSQL Python adapter
Auth	python-jose	3.3.0	JWT encode/decode (HS256)
ML Framework	scikit-learn	1.6.0	RandomForestClassifier + metrics
Data Processing	pandas	2.2.3	Dataset loading and feature engineering
Numerics	NumPy	2.2.1	Array operations
Model Serialise	joblib	1.4.2	Save/load trained model artifact
Testing	pytest + httpx	8.3.4	Backend API test suite
Containers	Docker Compose	3.x	Full-stack local orchestration

8 · Recommended Next Milestones

Priority	Milestone	Detail
HIGH	Google OAuth / Gmail Sign-In	Replace email+password auth with Google Identity Services — eliminate
HIGH	Live Screenshots with Backend Data	Capture actual browser screenshots showing JSON responses in DevTo
HIGH	External Validation Dataset	Evaluate the trained model on a held-out, non-duplicate dataset (e.g. a s
MEDIUM	Symptom Vocabulary Expansion	Grow NLP aliases from 14 to 50+ canonical symptoms to bridge the gap
MEDIUM	LLM-Augmented Analysis	Integrate a Gemini API call in the analyze() path to generate richer, conte
LOW	HTTPS + Production Deployment	Move from localhost to a cloud host (Railway, Render, or GCP Cloud Ru

9 · Medical Disclaimer & Ethical Notice

ZEEVAN is an educational and demonstrative project. It is not a licensed medical device, does not constitute medical advice, and must not be used for clinical decision-making. All predictions are labelled with demo: true in the API response. The symptom-to-disease dataset is synthetic and has not undergone clinical validation. Users experiencing health concerns must consult a qualified medical professional.